

System Software

Davide Berti
Software Engineer
Developer Technical Services

Software Creation Process: environment variables

Shell variables can improve the development experience

Some suggestions:

```
export SYSROOT=/opt/holly/sysroot
export SYSROOTA=/opt/holly/sysroot/arm-linux
export SYSROOTI=/opt/holly/sysroot/i386-linux
export PATH=/opt/holly/toolroot/bin:$PATH
```

In Simulator:

```
export LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib
```



Software Creation Process: environment variables - bash functions

```
function dohxtek()
{
  LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib \
  /opt/holly/sysroot/i386-linux/usr/bin/hxtek
}

function exe
{
  echo ""
       LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib
\
  /opt/holly/sysroot/i386-linux/Applications/com.palm/${1}/${1}
}
```



Software Creation Process: build utilities

pkg-config is a helper tool used when compiling applications and libraries. It helps you insert the correct compiler options on the command line. This allows the developer to omit the hard-coding of values on where to find glib (or other libraries).

It is language-agnostic, so it can be used for defining the location of documentation tools, for instance.

```
PACKAGES = hxui-1.0 directfb sqlite3

… -L$(USRDIR)/lib/libid3 -lid3 `pkg-config --errors-to-stdout
        --libs --cflags $(PACKAGES)`
```

For Simulator:

```
export PKG_CONFIG_PATH=/opt/holly/sysroot/i386-
  linux/usr/lib/pkgconfig
```

For Device:

```
export PKG_CONFIG_PATH=/opt/holly/sysroot/arm-linux/usr/lib/pkgconfig
```



Software Creation Process:hxui-1.0.pc

```
prefix=/usr
exec_prefix=${prefix}
libdir=${exec_prefix}/lib
includedir=/usr/include

Name: HXUI
Description: Hx UI Library
Version: 0.0
Requires:
Libs:  -lhxui
Cflags: -I${includedir}/hx
```



Software Creation Process: hxi utilities

Hxify utility helps generate xml file that specifies certain parameters for an Application

```
hxify [-d displayName] [-f fullPath] [-o output] [-e exe] [-r
  res]   [-h]
  -i icon:              path to icon for use in the App Launcher
  -d displayName:       displayName may contain spaces if quoted.
  -f fullPath:          fullPath may contain parameters if quoted.
                        Goes verbatim into executablePath.
  -o outputPath:        Will create app.hxi in outputPath
                        directory
  -e exePath:           Will drop exe from cwd/exePath into
                        outputPath. Goes verbatim into exePath.
  -r resPath:           Will copy Resources directory resPath
                        Goes verbatim into resPath.
  -h                    Will output this help menu
```



Software Creation Process: sample app.hxi file

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Comment -->
<hx>
        <application icon="Resources/App_Icon.png">
                <display_name>
                        dtshxskeleton
                </display_name>
                <executablePath>
                        dtshxskeleton
                </executablePath>
        </application>
</hx>
```



Software Creation Process: Tying it all Together in the Makefile ..

```
#  Copyright 2007, All rights reserved
#  Palm, Inc.

#set the component name
COMPONENT   = dtshxskeleton
#set the installation domain for the application
DOMAIN      = com.palmsdk

#source files
SOURCES     = $(COMPONENT).c

#toolroot prefix
TOOLROOT   = /opt/holly/toolroot
#specify packages for pkg-config
PACKAGES    +=  hx-1.0 \
                hxui-1.0 \
                lite \
                directfb \
                glib-2.0
```



Software Creation Process: Tying it all Together in the Makefile .. Specify variables

```
#the app.hxi file, specify which architecture, specific #
#pkg-config

ifeq ($(MAKECMDGOALS),arm)
        HOLLY_ARCH = arm-linux
        PKG_CONFIG_PATH = $(TOOLROOT)/arm-linux/local/bin
else
        HOLLY_ARCH = i386-linux
        PKG_CONFIG_PATH = $(TOOLROOT)/i386-
linux/local/bin
endif

#specify compiler based on arch flag
CC=$(HOLLY_ARCH)-gcc
#specify extra CFLAGS
CFLAGS  += -g
```



Software Creation Process: Tying it all Together in the Makefile .. Specify Variables

```
#specify simple variables
SDK_BASE     = /opt/holly
SYS_BASE     = $(SDK_BASE)/sysroot
TOOL_BASE    = $(SDK_BASE)/toolroot
HOLLY_BASE   = $(SYS_BASE)/$(HOLLY_ARCH)
APP_BASE     = $(HOLLY_BASE)/Applications

#specify target dir for package files
TARGET_DIR   = $(APP_BASE)/$(DOMAIN)/$(COMPONENT)
#specify target
TARGET       = $(TARGET_DIR)/$(COMPONENT)
#specify package source for hxpkg-build tool
PACKAGE_DIR  = package/${COMPONENT}
#specify package results for hxpkg-build tool
PACKAGE_RESULTS_DIR = packageResults
```



Software Creation Process: Tying it all Together in the Makefile .. Compile

```
I386:
mkdir -p $(TARGET_DIR)
$(CC) -g $(SOURCES) -o $(TARGET)
`$(PKG_CONFIG_PATH)/pkg-config –cflags --libs
$(PACKAGES)`
$(TOOLROOT)/bin/hxify -i "Resources/Terminal.png" -o
$(TARGET_DIR) -f "$(COMPONENT)" -r Resources -d
"$(COMPONENT)" -e $(OBJ) -n "$(COMPONENT)"

arm:
mkdir -p $(TARGET_DIR)
$(CC) -g $(SOURCES) -o $(TARGET) `$(PKG_CONFIG_PATH)/pkg-
config –cflags --libs $(PACKAGES)`
$(TOOLROOT)/bin/hxify -i "Resources/Terminal.png" -o
$(TARGET_DIR) -f "$(COMPONENT)"
-r Resources –d "$(COMPONENT)" -e $(OBJ) -n
"$(COMPONENT)“
mkdir -p $(PACKAGE_DIR)/CONTROL
mkdir -p $(PACKAGE_RESULTS_DIR)
mkdir -p $(PACKAGE_DIR)/data
```



Software Creation Process: Tying it all Together in the Makefile .. Build Package

```
cp -R CONTROL ${PACKAGE_DIR}
cp -R $(TARGET_DIR)/* $(PACKAGE_DIR)
$(HOLLY_BASE)/bin/hxpkg-build `pwd`/$(PACKAGE_DIR)
`pwd`/$(PACKAGE_RESULTS_DIR)
```

The purpose of the package is to facilitate loading applications onto the device. Based on the packaging system ipkg which, in turn, is based on the debian packager.



Software Creation Process: Package Specifics

The CONTROL directory

- holds the meta-data for the package.
- the control file is an ascii file fixed by the application developer

Currently, the following files are expected to be in the CONTROL directory

- control The basic repository of information.
- eula The end-user's licensing agreement.
- installed_size amount of space the application will take on the device.
- logo.png (optional) The icon displayed during the installation process.



```
int main (int argc, char* argv[])
{

  /* add a full-screen container as our drawing area */
  HxRectangle_t rect, bounds;
  /* initialize the UI toolkit */
  result = HxUiInit(&AppWindow, APP_NAME);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* open the resource file */
  result = HxResFileOpen(APP_RES_FILE, &ResFile);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* register cleanup function */
  atexit(AtExit_UserInterface);
```



```
int main (int argc, char* argv[])
{

  /* add a full-screen container as our drawing area */
  HxRectangle_t rect, bounds;
  /* initialize the UI toolkit */
  result = HxUiInit(&AppWindow, APP_NAME);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* open the resource file */
  result = HxResFileOpen(APP_RES_FILE, &ResFile);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* register cleanup function */
  atexit(AtExit_UserInterface);
```



```
int main (int argc, char* argv[])
{

  /* add a full-screen container as our drawing area */
  HxRectangle_t rect, bounds;
  /* initialize the UI toolkit */
  result = HxUiInit(&AppWindow, APP_NAME);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* open the resource file */
  result = HxResFileOpen(APP_RES_FILE, &ResFile);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* register cleanup function */
  atexit(AtExit_UserInterface);
```



main continued

```
HxResult result = 0;

/* enter event loop */
result = HxWindowEventLoop(AppWindow, 0);
HX_ASSERT(HX_OK == result);

/* Close up the UI toolkit & close application window. */
result = HxUiRelease();
```



Coding to the Falcon API: Message domains

`kHxActionMessageClass`	Action class (widgets)
`kHxInputMessageClass`	Input class (mouse, keyboard, wheel)
`kHxNotificationMessageClass`	Notification messages
`kHxSystemMessageClass`	System messages
`kHxSyncMessageClass`	Data synchronization messages
`kHxPowerMessageClass`	System power messages
`kHxWiFiMessageClass`	System WiFi messages
`kHxUserMessageClass`	User defined messages

Most events handled as messages.



HxMessageClass: Action

Of particular importance to Application Developers is the Action Message

```
enum {
  kHxAnyActionMessage = 0x0000,
  kHxCheckboxChanged,
  kHxRadiobuttonChanged,
  kHxCommandButtonChanged,
  kHxListSelected,
  kHxMenuSelected,
  kHxGadgetSelected,
  kHxScrollbarChanged,
  kHxWidgetGotFocus,
  ..
};
```



HxMessageClass: Action continued ..

Of particular importance to Application Developers is the Action Message

```
enum {
    kHxDropdownBoxOpened,
    kHxColumnHeaderChanged,
    kHxStorageListChanged,
    kHxDialogAboutToShow,
    kHxDialogAboutToHide,
    kHxStorageListReselected,
    kHxFileListSortChanged,
    ...
};
```



HxMessageClass: Notification

Also of note is the Notification Message Domain

```
enum {
    kHxAnyNotificationMessage = 0,/**< Any notification message
  */
    kHxClipboardReset,             /**< Clipboard was reset */
    kHxClipboardNewData,           /**< New data in clipboard */
    kHxClipboardDataAdded,         /**< Clipboard data was added
  */
    kHxApplicationIsActivated,     /**< Application switched in */
    kHxApplicationIsDeactivated,   /**< Application switched out
  */
    kHxApplicationIsShutdown,      /**< Application is shut down
  */
    kHxApplicationSaveState,       /**< Application save state */
    kHxApplicationMIMEAction,      /**< MIME data action */
    …
};
```



```
HxMessageListener_t       *pMenuListener = NULL;
static HxResult MenuCallback(const HxMessage_t*, void*);
..

result = HxMessageNewListener(&pMenuListener);
HX_EXIT_IF_ERROR(HX_OK == result, result, error);

/* listen for menu item selection */
result = HxMessageListen(pMenuListener, kHxActionMessageClass,
           kHxMenuSelected, MenuCallback, NULL);
```







- Events are handled by listeners.
- A single listener can listen for multiple messages.
- Each message instantiates 1 callback.

```
HxResult MenuCallback(const HxMessage_t* pMessage, void* pData)
{
    int32_t           id, size;
    HxCommandButton_t *pButton = NULL;
    HxResult          result = HX_OK;

    HX_ASSERT(NULL != pMessage);

    result = HxMessageGetData(pMessage, (void*)&id,  &size);
    HX_ASSERT(HX_OK == result);
    ..
    else if(id == RESID(About))
    {
        ..
    }
    return result;
```

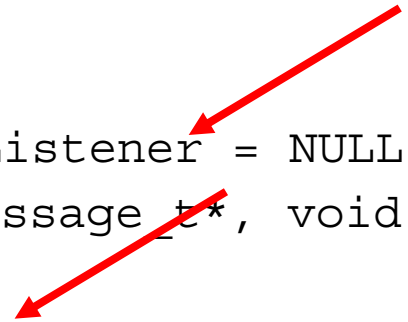


Coding to the Falcon API: Messages:
`kHxUserMessageClass`

- Developers can send their own messages
  - HxMessageSend
    - This function sends a message synchronously, any registered callbacks are immediately triggered. This type of messaging should only be used in the context of the same thread, for example in the default user interface thread.
    - Can’t be used between threads
  - HxMessagePost
    - This function posts a message asynchronously, the message is queued and the callbacks are triggered later. This type of messaging is useful for thread-to-thread communication, and for similar instances where the context wherefrom the message sent is different from the context where it is received.
    - Must be used between threads.



Define a listener, callback



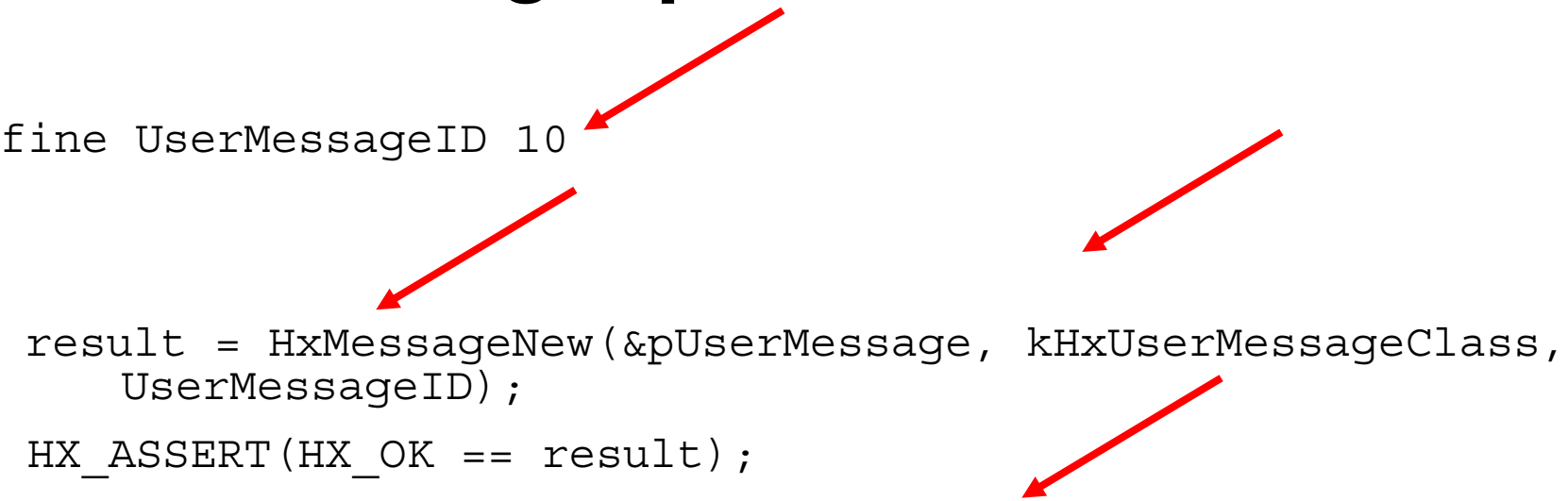
```
HxMessageListener_t       *spUserMessaggeListener = NULL;
HxResult       MyMessageHandler(const HxMessage_t*, void *);

HxResult MyMessageHandler(const HxMessage_t* message, void *data)
{
    HxResult        result = HX_OK;
    uint32_t        pSelectButton = 0;

    result = HxAlert(kHxWidgetAutoID, kHxAlertStyleOK, NULL, \
  "Received Message","Received User Message", &pSelectButton);
    HX_ASSERT(result == HX_OK);

    HxMessageDelete(pUserMessage);
    pUserMessage = NULL;
    return result;
}
```

Build a Message, post it!



```
#define UserMessageID 10
…

    result = HxMessageNew(&pUserMessage, kHxUserMessageClass,
        UserMessageID);
    HX_ASSERT(HX_OK == result);

    result = HxMessagePost(NULL, pUserMessage, 0);
    HX_ASSERT(HX_OK == result);

    return result;
}
```

Mapping Callbacks for a particular widget

A message callbacks is the normal way an application reacts to user actions. This is the recommended way to respond to events, rather than using the HxWidgetSetCallback API, which replaces the low-level event handler.

This function associates a message callback with a particular message action, such as a command button press, or a menu selection.



```
/* listen for menu messages */
result = HxWidgetMapMessageCallback((HxWidget_t*)spRootMenu,
            kHxActionMessageClass, kHxMenuSelected,
            MenuCallback, NULL);
```

Retrieve a handle to the ResFile

```
int main (int argc, char* argv[])
{

  /* initialize the UI toolkit */
  result = HxUiInit(&AppWindow, APP_NAME);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* open the resource file */
  result = HxResFileOpen(APP_RES_FILE, &spResFile);
  HX_EXIT_IF_ERROR(HX_OK == result, result, error);

  /* register cleanup function */
  atexit(AtExit_UserInterface);
```



RESID Macro

```
/* routine to convert a string to a resource ID */
uint32_t LookupResID(const char *idString)
{
    uint32_t id = UINT32_MAX; /* won't match anything */
    HxResult result = HxResFileLookupID(spResFile,  idString,
  &id);
    if (result != HX_OK)
    {
        fprintf(stderr, APP_NAME ": Unknown resource ID
               string '%s'\n", idString);
        exit(1);
     }
     return id;
}
#define RESID(x)    (LookupResID(#x))
```



Load resources from the Resource File

```
/* setup the root menu for the app */
result = HxResFileNewRootMenu(ResFile, RESID(RootMenu),
      &RootMenu, NULL);
HX_EXIT_IF_ERROR(HX_OK == result, result, error);


rect.x = rect.y =50;
rect.w = rect.h = 200;


HxContainerNew(kHxWidgetAutoID, (HxWidget_t*)AppWindow, &rect,
         kHxContainerAttrHasFrame | kHxContainerRoundedBottom,
\         &spContainer);


HX_EXIT_IF_ERROR(HX_OK == result, result, error);


HxResFileLoadContainer(ResFile, RESID(AContainer), spContainer,
      NULL);
HX_EXIT_IF_ERROR(HX_OK == result, result, error);
```



Hxui xml file

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hxui SYSTEM "hxui_1.0.dtd">
<!--Copyright (c) 2006 Palm, Inc. or its subsidiaries.
All rights reserved. -->

<hxui version="1.0" default_locale="en_US" display="1024">
  <root_menu id="RootMenu">
    <submenu id='$AUTO'>
      <text>File</text>
      <menu_item id="About" shortcut='A'>
        <text>About hxskeleton...</text>
      </menu_item>
      ..
    </submenu>
  </root_menu>
```



XML relative positioning

```
<dialog id="DialogAbout">
    <pos x="280" y="180"/><size w="240" h="115"/>
    <text>About hxApp</text>
    <label id="$AUTO">
        <pos x="10" y="10"/><size w="220" h="26"/>
        <text>hxApp</text>
        <font point_size="16" weight="bold" />
    </label>
    <label id="$AUTO">
        <pos x="$PREV:LEFT" y="$PREV:TOP+15"/><size w="220"
                h="16"/>
        <text>Version 1.0.1</text>
     </label>
    <command_button id="ButtonOpenOK">
        <pos x="7" y="81"/>
        <text>OK</text>
    </command_button>
</dialog>
```



Localizing the Application

- UTF-8 Character set
- Currently support 2 languages:
  - Spanish: es
  - English: en_US
- Consider Widget dimensions/placement for the length of the text
- Define strings in resource files
- Save the file in the UTF-8 format in your favorite text editor



Localizing the Application

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hxui SYSTEM "hxui_1.0.dtd">
<!--Copyright (c) 2006 Palm, Inc. or its subsidiaries.
All rights reserved. -->

<hxui version="1.0" default_locale="es_ES" display="1024">
  <root_menu id="RootMenu">
    <submenu id='$AUTO'>
      <text>File</text>
      <menu_item id="About" shortcut='A'>
        <text>About hxskeleton...</text>
      </menu_item>
      ..
    </submenu>
  </root_menu>
```



HxProperty: hxpropertydomain.h

Properties are a system wide setting. A few examples:

```
kHxPropertyBatteryLevel INT: Battery Charge Level in Percent
kHxPropertyBatteryChargerInserted BOOL: Charger Plug status
kHxPropertyBatteryStatus INT: values from
kHxPropertyDomainSystemPath hx system path property domain
kHxPropertyBundlePath STRING: Application bundle path
kHxPropertyResourcePath STRING: Application resource dir. path
kHxPropertyPreferencePath STRING: Application preference path
kHxPropertyDotConfigPath STRING: Per-user config file path
kHxPropertyScreenBitdepth INT: Current bitdepth of the screen
kHxPropertyScreenLidOpen BOOL: Lid is open?
kHxPropertyAudioVolume INT: Current audio volume
```



HxProperty: hxproperty.h

```
HxResult HxPropertyChanged (HxPropertyId_t id, HxPropertyIndex_t
  index)
```

Indicate a change of a property.

```
HxResult HxPropertyRegisterCallback ( HxPropertyId_t id,
  HxMessageCallback callback, void * data)
```

Register Callback for Property Change. Register a callback for specific property changes. There is no need to set up a listener, this API will handle that. You can register multiple callbacks for the same property change message.



Retrieve a System Property

```
result = HxPropertyGetString(kHxPropertyBundlePath, 0, \
       bundlestring, &bundlesize);
HX_ASSERT(HX_OK == result);


result = HxAlert(kHxWidgetAutoID, kHxAlertStyleOK, NULL,
  "Property Retrieval", bundlestring, &SelectedButton);
HX_ASSERT(HX_OK == result);
```



Software Creation Process: Shared Libraries

- Where to create and load application specific libraries?
  - In bundle folder with app
  - In domain folder
- Put in <domain> directory
  - Share with symbolic link or LD_LIBRARY_PATH.



Communicating with a TREO over BT

Communication takes place over a serial link from Falcon to a TREO. Applications on the Falcon and the TREO need to implement a serial manager protocol for this communication.

Falcon does not provide a transport layer. The “pipe” is a raw stream.

The Falcon does not have a Serial Manager component analogous to what one would find in Palm OS.

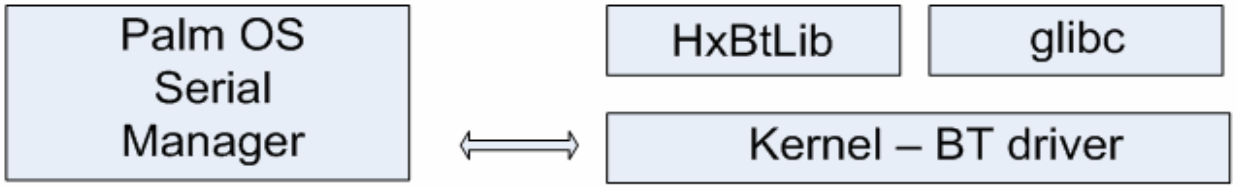


Falcon BT components vs. TREO BT components





Falcon



Falcon

- Use glibc open, read, write APIs for the data stream
- Use HxBtLib APIs to interact with the Driver, Radio, Discoverability

Treo

- Use the Serial Manager APIs (*Srm*)

TREO details

Applications on the TREO need to use the Serial Manager interface.

Do not leave the serial port open for extended periods of time. As a rule of thumb do not leave the serial port open for longer than 30 seconds.

Prevent automatic sleep if you do not want Palm OS to sleep while you are communicating.

- Use `EvtResetAutoOffTimer` periodically to facilitate this.

Clear errors, User `SrmClearErr` after an error from a previous Srm API.

Use a serial state machine model to design the transport protocol.



TREO Serial Manager Reference

Please refer to the PALM OS Developer Guide and SDK for more information about the Serial Manager.

A detailed example can also be found in:

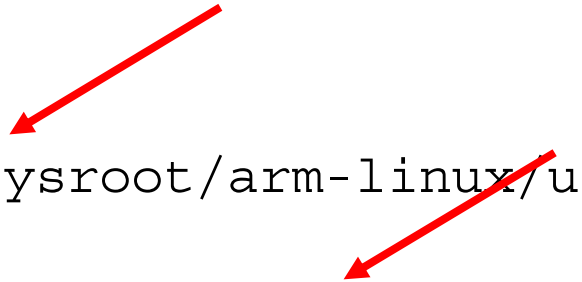
Palm OS Programming: The Developer's Guide, 2nd Edition [ILLUSTRATED] (Paperback)
by Neil Rhodes, Julie McKeeban



Build considerations

Include the header file HxBtLib.h in your application.

The Hx Bluetooth component does not have a pkg-config file specified. Please include the hx include directory and HxBtLib directly in your Makefile/IDE.



```
-I/opt/holly/sysroot/arm-linux/usr/include/hx/hxbt

-L$(USRDIR)/lib -lHxBtLib
```

Pairing

Programmatic Pairing vs. Pairing through the Startup Wizard

- Your application can use the System's Pairing dialogues to facilitate pairing. When the device is first boot, a Setup Wizard is launched that configures Locale, Date/Time information and optionally walks the user through a Phone Pairing wizard. It's also possible to pair from the System Preferences application -> Connections Option.
- Your application can make use of the Falcon APIs and attempt to programmatically pair with a TREO.
- We encourage developers to route an application through the System Preferences/Pairing Wizard when possible.



Falcon details

Falcon APIs are available that manipulate the Bluetooth driver:

```
BtErr_t HxBtLibOpen(BtHandle *handle_out)

BtErr_t HxBtOpen(BtHandle handle, UInt32 flags, BtAddr_t
  *addr, \ BtConnSetup_t *setup_opts, UInt16 *num_conns,
  BtConn_t *conns)

BtErr_t HxBtMakeDiscoverable(BtHandle handle)

bool_t HxBtIsDiscoverable(BtHandle handle)

BtErr_t HxBtPair(BtHandle handle, const BtAddr_t *addr, const
  char *pin, BtPairDoneFn_t pair_done, UserData_t data)

BtErr_t HxBtRemoveDevFromPairedDB(BtHandle handle, BtAddr_t
  *addr, bool_t all)

BtErr_t HxBtTurnOnRadio(BtHandle handle)

BtErr_t HxBtSetupProfile(BtHandle handle, const BtProfSetup_t
  *pBtProfSetup)
```



Falcon BT Programming Essentials

Open the library

```
err = HxBtLibOpen( &btHandle);
```

Verify the Radio is on

```
err = HxBtIsRadioOn (BtHandle handle)
err = HxBtTurnOnRadio (BtHandle handle);
```

Open the BT Device (success indicates the service was opened on the TREO)

```
pDevName = "/dev/ttyBt0";
// open the bluetooth serial port
fd = open( pDevName, O_RDWR | O_NOCTTY);
```



Falcon Serial Communication

Set options for the serial port

```
if ( tcgetattr( fd, &sTerm) < 0) {
        fprintf(stderr, "Failed to get port
  attributes.\n");
    }
cfmakeraw(&sTerm);
sTerm.c_lflag = 0;
sTerm.c_cflag &= ~CBAUD;
sTerm.c_cflag |= B115200 | CS8 | CRTSCTS;
```



Falcon Serial Communication

- get and set terminal attributes, line control
- get and set baud rate
- get and set terminal foreground process group ID

